

Relazione di Progetto: Gestore Aula Studio

Sistema di Gestione degli Accessi e delle Prenotazioni di un'Aula Studio
Domenico Gerardo Salvati - Mathias Pio Toscano

Ambiente di Sviluppo

Il progetto è stato sviluppato, testato e collaudato in ambiente Windows avvalendosi del sottosistema WSL (Windows Subsystem for Linux). La compilazione automatica del progetto è stata gestita tramite un file Makefile configurato in modo tale che, per compilare l'intero programma, sia sufficiente lanciare esclusivamente il comando `make` sul terminale. Il codice è stato scritto rispettando rigorosamente lo standard C99 e compilato su GCC con l'attivazione dei flag di controllo diagnostico `-Wall -pedantic` per garantire la massima pulizia, sicurezza e aderenza agli standard accademici.

1. Motivazione della Scelta dell'ADT

Per la realizzazione del Gestore dell'Aula Studio, è stata analizzata a fondo la richiesta e si è optato per l'utilizzo di specifici Tipi di Dato Astratti (ADT). Questo ha permesso di gestire in modo efficiente i posti a sedere, la lista d'attesa e lo storico delle prenotazioni, ottimizzando i tempi di esecuzione per le operazioni più frequenti:

- **Array Dinamico (Posti a sedere):** I posti fisici all'interno dell'aula sono stati modellati tramite un array di puntatori a struct `Studente`, allocato dinamicamente in base alla capienza inserita all'avvio del programma. Questa scelta garantisce un accesso diretto all'indice del posto in tempo costante $O(1)$, semplificando notevolmente le operazioni di assegnazione e verifica di disponibilità.
- **Tabella Hash (Ricerca e Tracciamento):** Per verificare in tempo reale chi è fisicamente presente in aula ed evitare ingressi duplicati della stessa matricola, erano necessarie ricerche estremamente veloci. È stata implementata una Tabella Hash basata sull'algoritmo *djb2*. Per gestire le collisioni in modo sicuro è stata usata la tecnica del concatenamento separato tramite liste collegate. In questo modo, l'ingresso, il check-out

e la ricerca avvengono con una complessità temporale media pari a $O(1)$.

- **Coda FIFO (Lista d'attesa):** Gli studenti che tentano l'accesso quando l'aula è piena vengono inseriti in una Coda. Trattandosi di una struttura *First-In, First-Out*, garantisce in modo naturale il principio di equità richiesto dal problema: il primo studente a essere messo in attesa sarà il primo a subentrare automaticamente quando verrà registrata un'uscita.
- **Lista Concatenata (Registro Prenotazioni):** Lo storico delle pratiche di prenotazione è memorizzato in una lista semplicemente concatenata. Poiché il numero di prenotazioni future non è noto a priori, questa struttura offre flessibilità dimensionale, permettendo inserimenti dinamici in $O(1)$ ed evitando continue e pesanti operazioni di riallocazione della memoria.

2. Progettazione e Architettura del Sistema

Il codice è stato suddiviso in moduli software indipendenti (file .h e .c) per garantire l'incapsulamento dei dati e una netta separazione delle responsabilità logiche. Le interazioni tra i componenti avvengono in modo gerarchico e controllato:

- **Modulo Studente:** Gestisce l'anagrafica utente. I campi interni sono nascosti al main tramite un puntatore opaco a struct. Utilizza funzioni sicure come strncpy in fase di inizializzazione per prevenire vulnerabilità nella memoria.
- **Modulo Prenotazione:** Associa un puntatore a struct Studente a una data, una fascia oraria e un posto specifico. Espone funzioni interne per validare formalmente e logicamente le stringhe di input (verificando formati e coerenza temporale/calendario).
- **Modulo Coda e Item:** La lista d'attesa è stata implementata introducendo un livello di astrazione tramite il file item.h (typedef Studente item;). Questo rende l'ADT Coda completamente generico e riutilizzabile: modificando un solo file, la struttura potrebbe accodare qualsiasi altro tipo di dato in futuri ampliamenti.
- **Modulo Aula:** Rappresenta il nucleo centrale logico del sistema. Coordina e gestisce tutte le strutture dati sottostanti (Hash, Coda, Array, Registro Storico). Implementa la logica di controllo, valutando matematicamente le intersezioni tra fasce orarie in base ai minuti effettivi per prevenire doppie prenotazioni sul medesimo posto.
- **Interfaccia Utente (Main):** Gestisce l'interazione a riga di comando (CLI). Cattura l'input dell'utente validandolo sistematicamente tramite la pulizia dei dati (getchar()), prevenendo

loop infiniti o interruzioni anomale dovute a errori di digitazione.

- **Suite di Test (Main Test):** Modulo eseguibile isolato e indipendente dal flusso d'uso, impiegato per la verifica automatizzata di tutte le strutture dati e della logica interna.

3. Come Usare e Interagire con il Sistema

Il software è interamente guidato da un'interfaccia a riga di comando (CLI) facile ed intuitiva, gestita a menù multipli. Per avviare il sistema occorre aprire il terminale nella cartella principale del progetto ed eseguire in sequenza:

1. Esclusivamente il comando `make` per compilare in automatico il codice sorgente e generare l'eseguibile.
2. Il comando `./gestore_aula` per eseguire l'applicativo principale.

All'avvio, il programma inizializza un'aula da 50 posti e presenta un menù principale numerato. L'utente digita il numero corrispondente all'azione desiderata (Ingresso diretto, Uscita, Stampa Stato o accesso al Sottomenù delle Prenotazioni) e preme Invio. Il software guida l'inserimento richiedendo i dati necessari e avvisando l'utente in merito ai formati richiesti (es. Date nel formato "GG/MM/AAAA" e orari nel formato "HH:MM-HH:MM").

Al termine di ogni transazione, il sistema stampa messaggi a schermo (es. `>>> SUCCESSO`, `>>> ERRORE` o `>>> AVVISO CRONOLOGICO`) per confermare in modo inequivocabile l'esito dell'operazione. È costantemente disponibile l'opzione "3" nel menù per generare in qualsiasi momento una fotografia in tempo reale delle occupazioni fisiche e della coda. Nel sottomenù, l'opzione "4" genera il report statistico completo che separa gli accessi effettivi dalle assenze per ogni fascia oraria.

4. Specifica Sintattica e Semantica

Di seguito è documentata rigorosamente l'architettura esposta dalle interfacce (file `.h`) dei quattro Moduli Principali. Per ogni operazione (per un totale di 32 funzioni) vengono indicati firma, semantica, pre/post-condizioni, output ed effetti collaterali.

4.1 Modulo Studente (`studente.h`)

- **Studente** `studente_crea(char* matricola, char* nome, char* corso);`

Semantica: Alloca dinamicamente la memoria per un nuovo studente e ne inizializza i

campi copiando le stringhe fornite.

Pre-condizioni: I parametri devono essere stringhe valide (non NULL) e correttamente terminate.

Post-condizioni: Viene creata una nuova struct `Studente`. I valori interni sono copie esatte dei parametri passati.

Valore di ritorno: Puntatore al nuovo studente creato, o NULL in caso di errore.

- **`void studente_distruggi(Studente* s);`**

Semantica: Libera la memoria allocata dinamicamente per la struct `studente` e imposta il puntatore a NULL.

Pre-condizioni: L'indirizzo 's' non deve essere NULL e '*s' deve puntare a un record valido.

Post-condizioni: Memoria liberata. Puntatore impostato a NULL per prevenire puntatori non validi (dangling pointers).

Valore di ritorno: Nessuno (void).

- **`char* studente_ottieni_matricola(Studente s);`**

Semantica: Fornisce l'accesso protetto in sola lettura alla stringa della matricola.

Pre-condizioni: Lo studente 's' deve essere un puntatore valido.

Post-condizioni: Lo stato interno dello studente non subisce alcuna modifica.

Valore di ritorno: Puntatore alla stringa della matricola, o NULL se 's' è invalido.

- **`void studente_stampa(Studente s);`**

Semantica: Stampa a video i dettagli formattati (Matricola, Nome, Corso).

Pre-condizioni: Il puntatore a struct `Studente` 's' deve essere allocato e valido.

Post-condizioni: Stato invariato. Output inviato su canale standard.

Valore di ritorno: Nessuno (void).

4.2 Modulo Prenotazione (`prenotazione.h`)

- **`Prenotazione prenotazione_crea(Studente s, char* data, char* fascia, int posto_scelto);`**

Semantica: Alloca e inizializza una struct `Prenotazione` assegnando studente, data, orario e numero di sedia.

Pre-condizioni: Il parametro 's' deve essere un puntatore valido. 'data' e 'fascia' non devono essere NULL.

Post-condizioni: Creata nuova prenotazione pronta per il sistema.

Valore di ritorno: Puntatore alla Prenotazione, o NULL in caso di errore.

- **void prenotazione_distruggi(Prenotazione* p);**

Semantica: Dealloca la struct Prenotazione dalla memoria.

Pre-condizioni: p è un doppio puntatore valido a una struct Prenotazione.

Post-condizioni: Memoria liberata, puntatore a NULL. Lo studente associato NON viene distrutto.

Valore di ritorno: Nessuno (void).

- **Studiante prenotazione_ottieni_studente(Prenotazione p);**

Semantica: Restituisce il riferimento allo studente associato alla prenotazione.

Pre-condizioni: p è un puntatore valido.

Post-condizioni: Nessuna modifica alla struct.

Valore di ritorno: Puntatore alla struct Studente.

- **void prenotazione_effettua_checkin(Prenotazione p);**

Semantica: Aggiorna lo stato della prenotazione confermando l'ingresso in aula.

Pre-condizioni: p è un puntatore valido.

Post-condizioni: Lo stato interno della prenotazione passa a 1 (confermato).

Valore di ritorno: Nessuno (void).

- **int prenotazione_ottieni_stato(Prenotazione p);**

Semantica: Legge lo stato attuale della prenotazione.

Pre-condizioni: p è un puntatore valido.

Post-condizioni: Nessuna modifica.

Valore di ritorno: Intero rappresentante lo stato (0 = Prenotato, 1 = Check-in), -1 errore.

- **int prenotazione_ottieni_posto(Prenotazione p);**

Semantica: Restituisce il numero del posto a sedere riservato.

Pre-condizioni: p è un puntatore valido.

Post-condizioni: Nessuna modifica.

Valore di ritorno: Intero indicante il numero di posto, -1 in errore.

- **char* prenotazione_ottieni_data(Prenotazione p);**

Semantica: Restituisce la stringa contenente la data della prenotazione.

Pre-condizioni: p è un puntatore valido.

Post-condizioni: Nessuna modifica alla struct.

Valore di ritorno: Puntatore alla stringa della data.

- **char* prenotazione_ottieni_fascia(Prenotazione p);**

Semantica: Restituisce la stringa contenente la fascia oraria della prenotazione.

Pre-condizioni: p è un puntatore valido.

Post-condizioni: Nessuna modifica alla struct.

Valore di ritorno: Puntatore alla stringa della fascia oraria.

- **void prenotazione_stampa(Prenotazione p);**

Semantica: Mostra su standard output i dettagli formattati della prenotazione.

Pre-condizioni: p è un puntatore valido.

Post-condizioni: Nessuna modifica.

Valore di ritorno: Nessuno (void).

- **int valida_data(const char* data);**

Semantica: Verifica che la stringa rispetti il formato "GG/MM/AAAA".

Pre-condizioni: 'data' deve essere puntatore a stringa valida e non NULL.

Post-condizioni: Lo stato del sistema e la stringa rimangono inalterati.

Valore di ritorno: 1 se formato e logica del calendario sono rispettati, 0 altrimenti.

- **int valida_fascia(const char* fascia);**

Semantica: Verifica che la stringa rispetti il formato "HH:MM-HH:MM".

Pre-condizioni: 'fascia' deve essere stringa valida e non NULL.

Post-condizioni: Nessuna modifica.

Valore di ritorno: 1 se formato e l'ora di uscita è logicamente successiva all'ingresso, 0 altrimenti.

4.3 Modulo Coda (coda.h)

- **Coda coda_crea(void);**

Semantica: Alloca e inizializza una nuova istanza di coda vuota.

Pre-condizioni: Nessuna.

Post-condizioni: Viene creata una coda vuota allocata dinamicamente.

Valore di ritorno: Puntatore alla nuova Coda, oppure NULL se la memoria è satura.

- **int coda_vuota(Coda q);**

Semantica: Verifica se l'istanza è priva di elementi.

Pre-condizioni: 'q' deve essere puntatore valido.

Post-condizioni: Stato inalterato.

Valore di ritorno: 1 se vuota, 0 se contiene elementi, -1 se il puntatore non è valido.

- **int coda_inserisci(item val, Coda q);**

Semantica: Accoda un nuovo elemento in fondo alla struttura (logica FIFO).

Pre-condizioni: 'q' valido e 'val' diverso da NULLITEM.

Post-condizioni: Dimensione incrementa. 'val' diventa l'ultimo elemento.

Valore di ritorno: 1 successo, 0 errore allocazione, -1 input invalidi.

- **item coda_estrai(Coda q);**

Semantica: Rimuove l'elemento situato in testa alla coda e lo restituisce.

Pre-condizioni: 'q' valida e non vuota.

Post-condizioni: Dimensione decresce. Il secondo elemento diventa la nuova testa.

Valore di ritorno: L'elemento estratto, o NULLITEM in caso d'errore.

- **void coda_distruggi(Coda* q);**

Semantica: Svuota interamente la coda liberando i nodi residui e il descrittore.

Pre-condizioni: L'indirizzo 'q' non deve essere NULL.

Post-condizioni: Memoria restituita al sistema. Puntatore impostato a NULL.

Valore di ritorno: Nessuno (void).

- **void coda_stampa(Coda q);**

Semantica: Scorre la coda dalla testa al fondo stampando gli elementi.

Pre-condizioni: 'q' puntatore valido.

Post-condizioni: Stato inalterato.

Valore di ritorno: Nessuno (void).

- **int coda_lunghezza(Coda q);**

Semantica: Restituisce il numero di elementi presenti.

Pre-condizioni: 'q' puntatore valido.

Post-condizioni: Stato inalterato.

Valore di ritorno: Numero intero di elementi, 0 se vuota/invalida.

4.4 Modulo Aula (aula.h)

- **Aula aula_crea(char* nome, int capienza_massima);**

Semantica: Alloca la struct Aula, configurando l'array dei posti, coda d'attesa, Hash e registro.

Pre-condizioni: 'nome' stringa valida, 'capienza_massima' intero maggiore di zero.

Post-condizioni: Creata nuova struct Aula inizializzata.

Valore di ritorno: Puntatore alla nuova Aula.

- **void aula_distruggi(Aula* a);**

Semantica: Rilascia l'intera memoria, distruggendo Coda, Hash e prenotazioni in sospeso.

Pre-condizioni: 'a' non nullo, puntante ad un'Aula valida.

Post-condizioni: Memoria liberata. Puntatore a NULL.

Valore di ritorno: Nessuno (void).

- **int aula_ingresso(Aula a, Studente s, int posto_richiesto);**

Semantica: Gestisce l'ingresso assegnando posto o spostando l'utente in attesa.

Pre-condizioni: 'a' ed 's' validi. 'posto_richiesto' nel limite della capienza.

Post-condizioni: Studente seduto o in coda.

Valore di ritorno: 1 (seduto), 2 (coda), 0 (errore).

- **int aula_uscita(Aula a, char* matricola_da_cercare);**

Semantica: Rimuove uno studente dall'aula tramite matricola in $O(1)$.

Pre-condizioni: 'a' e 'matricola' non NULL.

Post-condizioni: Studente rimosso, subentra il primo in coda automaticamente.

Valore di ritorno: 1 (successo), 0 (studente non presente).

- **void aula_stampa_stato(Aula a);**

Semantica: Mostra a schermo occupazioni e coda d'attesa.

Pre-condizioni: 'a' valido.

Post-condizioni: Nessuna modifica interna.

Valore di ritorno: Nessuno (void).

- **int aula_aggiungi_prenotazione(Aula a, Prenotazione p);**

Semantica: Registra la prenotazione dopo validazione conflitti orari (prevenzione doppie prenotazioni).

Pre-condizioni: 'a' e 'p' validi.

Post-condizioni: Inserimento nello storico.

Valore di ritorno: 1 (successo), 0 (conflitto o errore).

- **int aula_checkin_prenotazione(Aula a, char* matricola);**

Semantica: Convalida l'arrivo dello studente facendolo sedere.

Pre-condizioni: 'a' e 'matricola' validi.

Post-condizioni: Stato prenotazione aggiornato e studente inserito in aula.

Valore di ritorno: 1 (seduto), 2 (in coda), -1 (già entrato), 0 (non trovato).

- **int aula annulla_prenotazione(Aula a, char* matricola);**

Semantica: Rimuove una prenotazione in sospeso e dealloca lo studente.

Pre-condizioni: 'a' e 'matricola' validi.

Post-condizioni: Nodo storico svuotato e rimosso dalla lista concatenata.

Valore di ritorno: 1 (successo), 0 (fallimento/già in aula).

- **void aula_stampa_report_prenotazioni(Aula a);**

Semantica: Genera il report statistico degli accessi e delle assenze.

Pre-condizioni: 'a' valido.

Post-condizioni: Nessuna.

Valore di ritorno: Nessuno (void).

- **int aula_studente_esiste(Aula a, char* matricola);**

Semantica: Controlla la presenza di una matricola tramite ricerca Hash $O(1)$.

Pre-condizioni: 'a' e 'matricola' validi.

Post-condizioni: Nessuna.

Valore di ritorno: 1 (presente), 0 (assente).

5. Razionale dei Casi di Test

La fase di validazione del codice è stata demandata a un modulo eseguibile automatizzato (main_test.c). Il sistema isola ogni scenario all'interno di funzioni dedicate, valutando gli stati in memoria tramite macro standard della libreria <assert.h>. Il fallimento di un'asserzione causa l'arresto immediato del programma: raggiungere il termine dell'esecuzione certifica l'assenza di anomalie logiche. La test suite risolve puntualmente i casi considerati più importanti, e casi limite:

1. **Verifica della registrazione degli studenti (test_registrazione_studenti)**

- **Obiettivo:** Validare l'allocazione dinamica e il completamento dei dati anagrafici.
- **Input:** Chiamata al costruttore passandogli i parametri "MAT01", "Mario", "Informatica".
- **Oracolo:** Il costruttore deve restituire il puntatore allocato senza generare puntatori nulli.
- **Output:** L'asserzione su `s != NULL` viene superata con successo.

2. **Verifica dell'ingresso senza prenotazione (test_ingresso_senza_prenotazione)**

- **Obiettivo:** Verificare l'assegnazione spaziale diretta di un posto fisico.
- **Input:** Richiesta per l'ingresso e la seduta al posto fisico 0.

- **Oracolo:** La funzione deve registrare il dato nell'array interno e ritornare 1 (successo).
- **Output:** Lo stato dell'aula viene alterato, la funzione passa l'assert a 1.

3. Test della lista di attesa (test_lista_attesa)

- **Obiettivo:** Eseguire una simulazione di saturazione per innescare e validare l'ADT Coda.
- **Input:** Si forza la creazione di un'aula di capienza limitata a 1 inserendo un secondo studente.
- **Oracolo:** L'algoritmo non deve sovrascrivere l'array limitato, ma dirigere lo studente verso la coda restituendo 2.
- **Output:** Inserimento nella struttura FIFO convalidato.

4. Test dell'inserimento delle prenotazioni (test_inserimento_prenotazioni)

- **Obiettivo:** Archiviare in modo persistente una prenotazione all'interno della lista concatenata.
- **Input:** Istanziamento di una prenotazione lecita di Mario assegnata al posto 2.
- **Oracolo:** Modifica dei puntatori in testa del registro storico con restituzione del valore 1.
- **Output:** Record immagazzinato correttamente nella memoria storica, superando l'assert.

5. Verifica disponibilità e blocco conflitti (test_disponibilita_posti)

- **Obiettivo:** Eseguire un test di robustezza della logica matematica (Prevenzione delle doppie prenotazioni).
- **Input:** Si tentano di registrare due prenotazioni sullo stesso posto fisico con le fasce "10:00-12:00" e "11:00-13:00".
- **Oracolo:** La prima transazione viene confermata (1), la seconda subisce un blocco (0) in virtù dell'intersezione.
- **Output:** Calcolo matematico riuscito. La seconda prenotazione non va a buon fine, tutelando l'integrità del posto.

6. Verifica annullamento e aggiornamento (test annullamento disponibilita)

- **Obiettivo:** Controllare lo stato dei collegamenti e la corretta deallocazione di un nodo in una lista concatenata.
- **Input:** Comando di annullamento prenotazione fornendo la matricola chiave "MAT08".
- **Oracolo:** L'algoritmo intercetta il nodo, lo scollega dalla catena, invoca free() e ritorna 1.
- **Output:** Nessun puntatore sospeso generato. La rimozione è completata correttamente.

7. Test del check-in e check-out (test_checkin_checkout)

- **Obiettivo:** Validare la transizione di stato di una risorsa (prenotazione -> convalida -> uscita).
- **Input:** Operazione in sequenza di Check-in e successiva Uscita per la medesima matricola "MAT09".
- **Oracolo:** Entrambe le chiamate devono interagire e aggiornare la Tabella Hash in tempo reale (restituendo 1).
- **Output:** Il ciclo completo è simulato positivamente, verificando i cambi di stato in tempo reale.

8. Test dello storico e dei report (test_storico_report)

- **Obiettivo:** Implementare una gestione sicura dei casi limite e delle eccezioni di runtime.
- **Input:** Invocazione delle funzioni di stampa su un'Aula appena creata, completamente vuota e senza dati.
- **Oracolo:** Il sistema deve superare e gestire i controlli di sicurezza a NULL senza causare alcun errore di accesso alla memoria (Segmentation Fault).
- **Output:** Il report esegue la stampa dei parametri a zero e il programma raggiunge intatto la fine dei test.